DEPLOYMENT & OPERATIONS MANUAL

Local AI Development Environment Setup

Complete Step-by-Step Deployment Guide: llama.cpp Backend — Aider & Crush Frontends — GPU Acceleration

Target OS: Arch Linux & Debian/Ubuntu

Hardware: CPU-only or hybrid CPU+GPU setups

Table of Contents

1. System Architecture Overview

2. Installing System Dependencies

3. Compiling llama.cpp from Source

4. Isolated Installation of Aider

5. Model Selection, Hugging Face Download & Hardware Scaling

6. Executable Commands & Shell Automation

7. Using llama-cli for On-Demand Queries

Setup Guide: Crush as an Alternative to Aider

- 1. Overview & Comparison
- 2. Installation
- 3. Configuration Setup
- 4. Running the Complete Stack
- 5. Shell Aliases & Automation

GPU Acceleration with CUDA

- Prerequisites: Installing the CUDA Toolkit
- Recompiling llama.cpp with CUDA Support
- Offloading Layers to GPU (-ngl)
- Storing Models on a Secondary Drive

Web Search Integration via Tool Proxy

- 1. Why the Tool Proxy Approach
- 2. Architecture Overview
- 3. How Tool Calling Works
- 4. Installing Dependencies
- 5. The Proxy Script
- 6. System Prompt & When the Model Searches
- 7. Connecting Aider and Crush to the Proxy
- 8. Shell Aliases & Full Stack Startup
- 9. Alternative Search Backends

Technical Concept Reference & FAQs

Performance Factors & Latency Troubleshooting

- Common Errors & Fixes

1. System Architecture Overview

This setup utilizes a decoupled client-server architecture designed for high efficiency and total local privacy:

- **Backend (`llama-server`):** Compiled natively from `llama.cpp` source code. It loads quantized LLM weights into system RAM and exposes an OpenAI-compatible HTTP REST API on port `8080`.
- **Frontend (`aider`):** A Git-integrated AI coding CLI tool. It inspects project files, formats edit prompts, communicates with the backend server, and directly applies modifications to your codebase.

> **NOTE**
>
> **Why compile `llama.cpp` from source instead of using a pre-built package?** Pre-built binaries distributed via package managers (e.g., `pacman`, `apt`) are compiled for a generic x86-64 target and cannot take advantage of CPU-specific instruction sets such as AVX2 or AVX-512. Compiling locally with CMake allows the toolchain to detect and use all instruction sets supported by your processor, resulting in noticeably faster prompt evaluation and token generation speeds. Additionally, enabling GPU support (CUDA) requires building with the `-DGGML_CUDA=ON` flag, which is not available in standard pre-built distributions.

---

2. Installing System Dependencies

To compile `llama.cpp` natively and prevent Python package conflicts (particularly on newer releases like Python 3.14), install the required build tools and package managers:

Arch Linux

```
sudo pacman -S base-devel cmake git python-uv
```

Debian / Ubuntu

```
sudo apt update
sudo apt install build-essential cmake git python3-pip

# Install astral-sh uv for Debian/Ubuntu
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.bashrc
```

> **NOTE**
>
> The commands above install `cmake` as part of the dependency bundle. Verifying your CMake version, resolving conflicts with existing installations, or upgrading CMake manually is outside the scope of this guide. If `cmake` is already present on your system, you can skip or adapt the relevant line. Refer to the official CMake documentation for any installation issues.

---

3. Compiling llama.cpp from Source

Clone the official repository and build the release binaries leveraging full CPU parallel processing:

```
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp

# Configure build system and compile natively
cmake -B build
cmake --build build --config Release -j$(nproc)
```

Upon completion, the server binary `llama-server` and CLI binary `llama-cli` will be available inside `build/bin/`.

---

4. Isolated Installation of Aider

To prevent system-level Python conflicts and avoid compilation failures of underlying C/Fortran libraries (such as `scipy` or `numpy` under newer Python runtimes), force an isolated Python 3.12 environment using `uv`:

```
# Install Aider inside an isolated Python 3.12 environment
uv tool install --python 3.12 aider-chat

# Ensure user binaries are present in your shell PATH
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc   # For Zsh (default on Arch Linux)
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc  # For Bash (default on Debian)

source ~/.zshrc || source ~/.bashrc
```

---

5. Model Selection, Hugging Face Download & Hardware Scaling

Finding GGUF Models on Hugging Face

The `llama.cpp` backend operates exclusively on GGUF formatted models. To locate them on Hugging Face: * Search for the model family name appended with GGUF (e.g., `Qwen3.8-27B GGUF`). * Rely on well-maintained quantization repositories from community creators such as `bartowski` or `TheBloke`.

Download Methods

- **Terminal / Automation (Recommended):** Fetch using `curl` or `wget`: `bash mkdir -p ~/models && cd ~/models curl -L -O https://huggingface.co/<model-repo>/resolve/main/Qwen3.8-27B-UD-Q4_K_M.gguf`
- **Manual Browser Download:** Download directly from the Hugging Face *Files and versions* tab into `~/models/`.

Hardware Scaling & Quantization Matrix

Model selection depends heavily on available System RAM or GPU VRAM. Quantization formats (e.g., `Q4_K_M`, `Q6_K`) compress weights while retaining accuracy.

System RAM	Recommended Model Size	Quantization	File Size	Context Window (`-c`) & RAM Usage
8 GB RAM	3B–7B Parameters	`Q4_K_M`	~2.5–4.0 GB	Limit to `-c 4096` to avoid memory swapping
16 GB RAM	7B–8B Parameters	`Q5_K_M` / `Q6_K`	~5.5–7.0 GB	Up to `-c 16384` (~1.5 GB allocated for KV Cache)
32 GB RAM	14B Parameters	`Q4_K_M` / `Q5_K_M`	~9.0–11.0 GB	Up to `-c 32768` (~3.0 GB allocated for KV Cache)
64 GB RAM	14B–32B Parameters	`Q6_K` / `Q8_0`	~12.0–20.0 GB	Optimal: `-c 32768` or `-c 65536` with total stability

---

6. Executable Commands & Shell Automation

Option A: Launching `llama-server` (Backend Service)

```
/path/to/llama.cpp/build/bin/llama-server   -m ~/models/Qwen3.8-27B-UD-Q4_K_M.gguf   -t $(nproc) -c 32768 --port 8080
```

Option B: Launching `aider` (CLI Client)

```
~/.local/bin/aider   --openai-api-base http://localhost:8080/v1   --openai-api-key not-needed   --model openai/qwen3
```

Option C: Native Interactive CLI Chat (Without Aider or HTTP Server)

```
/path/to/llama.cpp/build/bin/llama-cli   -m ~/models/Qwen3.8-27B-UD-Q4_K_M.gguf   -c 32768 -t $(nproc) -cnv
```

Recommended Shell Aliases

Add these aliases to your `~/.bashrc` or `~/.zshrc`:

```
alias llama-coder="/path/to/llama.cpp/build/bin/llama-server -m ~/models/Qwen3.8-27B-UD-Q4_K_M.gguf -t $(nproc) -c 32768 --port 8080"
alias aider-local="~/.local/bin/aider --openai-api-base http://localhost:8080/v1 --openai-api-key not-needed --model openai/qwen3"
```

---

7. Using `llama-cli` for On-Demand Queries

`llama-cli` is the lightweight alternative to the full server stack. It loads the model, answers your query, and exits — no persistent HTTP server required. This makes it ideal for quick one-off questions, scripting, or situations where you do not want to keep a server running in the background.

> **NOTE**
>
> **Trade-off:** Every invocation of `llama-cli` reloads the model from disk and reallocates the KV cache from scratch. If you plan to send multiple messages in sequence, the interactive mode (`-cnv`) is more efficient than re-launching the binary each time.

Interactive Conversation Mode

Start a persistent back-and-forth chat session in the terminal:

```
/path/to/llama.cpp/build/bin/llama-cli   -m ~/models/Qwen3.8-27B-UD-Q4_K_M.gguf   -t $
(nproc) -c 32768   -cnv
```

The `-cnv` flag enables conversation mode: the model retains the full chat history within the session and responds to each new input. Type your message and press Enter. To exit, send an empty line or press `Ctrl+C`.

Single-Shot Prompt (Non-Interactive)

Send a single prompt and get the output directly in the terminal — useful for scripting or piping output to other tools:

```
/path/to/llama.cpp/build/bin/llama-cli   -m ~/models/Qwen3.8-27B-UD-Q4_K_M.gguf   -t $
(nproc) -c 8192   -p "Write a Python function that merges two sorted lists."
```

The `-p` flag passes the prompt directly. The model prints its response to stdout and exits.

Piping a File as Input

Inject the contents of a file directly into the prompt — useful for asking the model to explain, refactor, or review a specific file:

```
echo "Explain what this code does:" | cat - myfile.py |   /path/to/llama.cpp/build/bin/
llama-cli   -m ~/models/Qwen3.8-27B-UD-Q4_K_M.gguf   -t $(nproc) -c 32768 -p "$(cat)"
```

A simpler approach using command substitution:

```
/path/to/llama.cpp/build/bin/llama-cli   -m ~/models/Qwen3.8-27B-UD-Q4_K_M.gguf   -t $
(nproc) -c 32768   -p "Refactor this Python code for readability:
$(cat myfile.py)"
```

With GPU Offloading

If you compiled `llama.cpp` with CUDA support, add `-ngl` to offload layers to the GPU and significantly reduce response latency:

```
/path/to/llama.cpp/build/bin/llama-cli   -m ~/models/Qwen3.8-27B-UD-Q4_K_M.gguf   -t $
(nproc) -c 32768   -ngl 20   -cnv
```

Useful `llama-cli` Parameters at a Glance

Parameter	Description
`-m <path>`	Path to the `.gguf` model file
`-t <n>`	Number of CPU threads (set to physical core count)
`-c <n>`	Context window size in tokens
`-p "<text>"`	Prompt string for single-shot mode
`-cnv`	Enable interactive conversation mode
`-ngl <n>`	Number of layers to offload to GPU (requires CUDA build)
`-n <n>`	Maximum number of tokens to generate per response
`--temp <f>`	Sampling temperature (e.g., `0.7`); lower = more deterministic
`--no-display-prompt`	Suppress echoing the input prompt in the output (useful for scripts)

Setup Guide: Crush as an Alternative to Aider

This guide covers how to install, configure, and use **Crush** by Charmbracelet as a modern, terminal-based AI coding assistant and alternative to Aider, integrated with a local `llama.cpp` backend.

---

1. Overview & Comparison

Feature	Aider	Crush
Developer / Ecosystem	Independent (`aider-chat`)	Charmbracelet (`charm.sh`)
UI / TUI Design	Classic CLI text-stream	Rich TUI powered by Bubble Tea & Lip Gloss
Backend Integration	LiteLLM / OpenAI API Protocol	Native / OpenAI-compatible provider schema
Configuration	Command-line flags, `.aider.conf.yml`	Declarative JSON configuration (`crush.json`)

---

2. Installation

Package Managers

macOS & Linux (Homebrew)

```
brew install charmbracelet/tap/crush
```

Arch Linux (AUR)

```
yay -S crush-bin
# or
paru -S crush-bin
```

Debian / Ubuntu

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://repo.charm.sh/apt/gpg.key | sudo gpg --dearmor -o /etc/apt/keyrings/
charm.gpg
echo "deb [signed-by=/etc/apt/keyrings/charm.gpg] https://repo.charm.sh/apt/ * *" | sudo
tee /etc/apt/sources.list.d/charm.list
sudo apt update && sudo apt install crush
```

Direct Download / Go Install

If you have Go (1.22+) installed, build directly from source:

```
go install github.com/charmbracelet/crush@latest
```

---

3. Configuration Setup

Crush uses a JSON schema located in `~/.config/crush/crush.json` to define custom models and local backends.

1. Create the configuration directory: `bash mkdir -p ~/.config/crush`
2. Create and edit the configuration file `~/.config/crush/crush.json`: `bash nano ~/.config/crush/crush.json`
3. Insert the following configuration structure:

```
{
  "$schema": "https://charm.land/crush.json",
  "providers": {
    "llama-local": {
      "id": "local-qwen3.8",
      "name": "local-qwen3.8",
      "type": "openai-compat",
      "base_url": "http://localhost:8080/v1",
      "api_key": "not-needed",
      "models": [
        {
          "id": "local-qwen3.8",
          "name": "Qwen 3.8 Local",
          "context_window": 32768,
          "default_max_tokens": 4096,
          "can_reason": true,
          "supports_tools": true
        }
      ]
    }
  }
}
```

Key parameter reference:

- **`type: "openai-compat"`** — Tells Crush to communicate with this provider using the OpenAI HTTP API protocol (i.e., `POST /v1/chat/completions`). This is the same protocol that `llama-server` exposes, making the two fully compatible without any translation layer.
- **`base_url`** — The root URL of the local backend. Crush appends `/chat/completions` to this automatically. Point this to wherever `llama-server` is running (`localhost` for local setups, or a remote IP/hostname for network deployments).
- **`api_key: "not-needed"`** — `llama-server` does not enforce authentication by default, but Crush requires this field to be non-empty. Any non-empty string works.
- **`context_window`** — Must match the `-c` value you passed to `llama-server` at startup. If Crush sends a prompt that exceeds this number of tokens, `llama-server` will truncate or error. Keep these values in sync.
- **`default_max_tokens`** — The maximum number of tokens Crush will request for each model response. This does **not** affect the total context window — it only caps the length of a single reply. Increase it if the model cuts off long responses.
- **`can_reason: true`** — Signals to Crush that this model supports extended reasoning (chain-of-thought) output, such as the thinking/non-thinking modes in Qwen3. Enables reasoning-specific UI features and prompt strategies within Crush.
- **`supports_tools: true`** — Indicates that the model can handle function/tool calling requests formatted as structured JSON. Required for Crush's agent capabilities (file operations, shell commands, etc.) to work correctly.

- Add the required environment variables to your shell configuration file (`~/.zshrc` or `~/.bashrc`):

`bash export OPENAI_API_BASE="http://localhost:8080/v1" export OPENAI_API_KEY="not-needed"`

Reload the shell configuration:

`bash source ~/.zshrc # or source ~/.bashrc`

- **`OPENAI_API_BASE`** — Specifies the endpoint Crush uses to route API calls to the model. Must match the `base_url` defined in `crush.json` and the `--port` value passed to `llama-server`.
- **`OPENAI_API_KEY`** — Required by Crush even when no authentication is enforced. For local setups where `llama-server` runs without an API key, use any non-empty string (e.g., `"not-needed"`).

---

4. Running the Complete Stack

Step 1: Launch the Local `llama.cpp` Backend

Ensure your `llama-server` is up and running on port `8080`:

```
/path/to/llama.cpp/build/bin/llama-server   -m ~/models/your-model.gguf   -t $(nproc) -c
32768 --port 8080
```

> The `-m` path and model ID in the `crush.json` config must refer to the same model. The example above and the JSON config in section 3 both target `Qwen3-8B`; replace the filename accordingly if you are using a different model.

Step 2: Start Crush

Launch Crush inside your project repository:

```
crush --provider llama-local --model local-qwen3.8
```

---

5. Shell Aliases & Automation

Add an alias to your `~/.bashrc` or `~/.zshrc` for quick access:

```
alias crush-local="crush --provider llama-local --model local-qwen3.8"
```

After updating your configuration (`source ~/.zshrc`), you can start coding with Crush simply by typing:

```
crush-local
```

GPU Acceleration with CUDA

By default, `llama.cpp` runs inference entirely on the CPU. If you have a CUDA-capable NVIDIA GPU, you can offload transformer layers to GPU VRAM using the `-ngl` flag. Even partial GPU offloading significantly reduces **Time To First Token (TTFT)** and improves overall throughput compared to a pure CPU setup.

Prerequisites: Installing the CUDA Toolkit

The NVIDIA CUDA Toolkit (`nvcc` compiler + runtime libraries) must be present on your system before recompiling `llama.cpp` with GPU support.

Arch Linux

The toolkit is packaged in the official Arch repositories and does not interfere with your existing NVIDIA display drivers:

```
sudo pacman -S cuda
```

Arch installs CUDA under `/opt/cuda`. Add the binaries to your shell environment by appending these lines to `~/.zshrc` or `~/.bashrc`:

```
export PATH="/opt/cuda/bin:$PATH"
export LD_LIBRARY_PATH="/opt/cuda/lib64:$LD_LIBRARY_PATH"
```

Reload your shell configuration:

```
source ~/.zshrc   # or source ~/.bashrc
```

Debian / Ubuntu

Use the official NVIDIA installer for your distribution from the CUDA Downloads page. Select the `deb (network)` method for the cleanest integration. After installation the toolkit is typically located at `/usr/local/cuda`.

Verifying the Installation

On any distribution, confirm that `nvcc` is accessible:

```
nvcc --version
```

Recompiling `llama.cpp` with CUDA Support

With the CUDA toolkit installed and `nvcc` in your `PATH`, remove the previous CPU-only build and recompile with the `GGML_CUDA` flag enabled:

```
cd llama.cpp

# Remove the previous CPU-only build directory
rm -rf build

# Configure CMake with CUDA support
cmake -B build -DGGML_CUDA=ON

# Compile using all available physical CPU cores
cmake --build build --config Release -j$(nproc)
```

The resulting `llama-server` and `llama-cli` binaries in `build/bin/` will now support GPU layer offloading via the `-ngl` parameter.

Offloading Layers to GPU (`-ngl`)

The `-ngl` parameter (also accepted as `--gpu-layers` or `--n-gpu-layers`) controls how many transformer layers are loaded into GPU VRAM. Any layers beyond that limit fall back to system RAM and CPU processing.

How Layer Offloading Works

A transformer model is not a monolithic block — it is composed of a stack of independent **transformer layers** (also called blocks or decoder layers), each containing the attention heads and feed-forward networks that do the actual computation. When `llama-server` loads a model, it can assign each layer individually to either the GPU or the CPU.

The key insight is that **inference is sequential**: the model processes one layer at a time, in order. If a layer lives in VRAM, computation happens at GPU speed (very fast). If a layer lives in RAM, the data must be transferred over the PCIe bus and processed by the CPU (much slower). This means even a partial offload yields a measurable speed gain, because the bottleneck layers (early and middle ones) get accelerated.

Layer Count Reference by Model Size

Model Size	Typical Layer Count	Example Models
3B parameters	28 layers	Qwen2.5-Coder-3B, Llama-3.2-3B
7B–8B parameters	32 layers	Llama-3.1-8B, Mistral-7B
8B (Qwen3 family)	**36 layers**	**Qwen3-8B** (8.2B params, reasoning + non-thinking modes)
14B parameters	48 layers	Qwen2.5-Coder-14B, Phi-4-14B
32B parameters	64 layers	Qwen2.5-Coder-32B
70B parameters	80 layers	Llama-3.1-70B, Llama-3.3-70B

> **NOTE**
> Layer counts vary between model families — Qwen3 models in particular use a different architecture than same-size Llama/Mistral variants. The exact value is always listed in the model's `config.json` on Hugging Face, or reported by `llama-server` at startup.

VRAM Consumption Estimation

Each layer's VRAM cost depends on the model size and quantization format. As a rough approximation:

```
VRAM per layer ≈ total_model_file_size_GB / total_layer_count
```

Practical examples:

Model	Quantization	File Size	Layers	VRAM per Layer	Full Offload VRAM
7B	Q4_K_M	~4.1 GB	32	~128 MB	~4.5 GB (+ overhead)
Qwen3-8B	**Q4_K_M**	**~5.2 GB**	**36**	**~144 MB**	**~6.0 GB (+ overhead)**
Qwen3-8B	**Q6_K**	**~6.7 GB**	**36**	**~186 MB**	**~7.5 GB (+ overhead)**
14B	Q4_K_M	~8.5 GB	48	~177 MB	~9.5 GB (+ overhead)
14B	Q6_K	~11.0 GB	48	~229 MB	~12.0 GB (+ overhead)
32B	Q4_K_M	~18.0 GB	64	~281 MB	~20.0 GB (+ overhead)

NOTE

The overhead (~0.5–1.5 GB) accounts for the embedding layer, the output projection, and the KV cache allocated for the context window.

Comparative Scenarios

The table below shows how `-ngl` affects performance and resource usage for a **14B Q4_K_M** model on a system with 8 GB VRAM:

`-ngl` Value	Layers on GPU	Layers on CPU	Approx. VRAM Used	Typical Token Generation Speed
`0`	0 / 48	48 / 48	~0 GB	Baseline (CPU-only)
`16`	16 / 48	32 / 48	~3.0 GB	~1.5–2× faster than baseline
`32`	32 / 48	16 / 48	~6.0 GB	~3–4× faster than baseline
`48`	48 / 48	0 / 48	~9.5 GB	Maximum speed — requires ≥10 GB VRAM

NOTE

Speed multipliers are indicative and vary with CPU, RAM bandwidth, and PCIe generation. The TTFT improvement is typically even more pronounced than token generation speed, since the entire prompt must be processed before the first token is produced.

How to Calibrate the Value

- **Start conservatively** (e.g., `-ngl 16`) and read the VRAM allocation from `llama-server`'s startup logs.
- **Increase incrementally** until you approach — but do not exceed — your GPU's memory limit. Exceeding it causes the process to crash or fall back silently.
- For a **full offload**, set `-ngl` to the model's total layer count (e.g., `-ngl 48` for a 14B model). Only attempt this if the model's file size fits within your available VRAM with some headroom for overhead.

Example command with partial GPU offloading:

```
/path/to/llama.cpp/build/bin/llama-server   -m ~/models/your-model.gguf   -t $(nproc)   -c
32768   -ngl 20   --port 8080
```

Updated shell alias:

```
alias llama-server-gpu="/path/to/llama.cpp/build/bin/llama-server -m ~/models/your-
model.gguf -t $(nproc) -c 32768 -ngl 20 --port 8080"
```

> **NOTE**
> The `-t` parameter should still be set to your physical CPU core count (not logical threads), as it governs the CPU threads used for any layers that remain on the CPU side.

Storing Models on a Secondary Drive

`.gguf` files are fully **self-contained** — no sidecar files or additional directories are created alongside them. They can be stored on any drive or mount point without affecting functionality.

To use a secondary storage device for your models:

```
# Create a dedicated directory on the secondary drive
mkdir -p /mnt/storage/models

# Download models directly to that location
cd /mnt/storage/models
curl -L -O https://huggingface.co/<repo>/<model>.gguf
```

Pass the full path via the `-m` flag and update your aliases to point to the new location:

```
alias llama-server-gpu="/path/to/llama.cpp/build/bin/llama-server -m /mnt/storage/models/
your-model.gguf -t $(nproc) -c 32768 -ngl 20 --port 8080"
```

Web Search Integration via Tool Proxy

This section explains how to give the local LLM the ability to search the internet in real time, while keeping **Aider and Crush as the primary entry points** with no changes to how you use them day to day.

---

1. Why the Tool Proxy Approach

`llama-server` exposes an OpenAI-compatible API but **does not execute tool calls** — it only generates the JSON that describes which tool should be called and with which arguments. Something external must intercept that output, run the actual search, and feed the results back to the model.

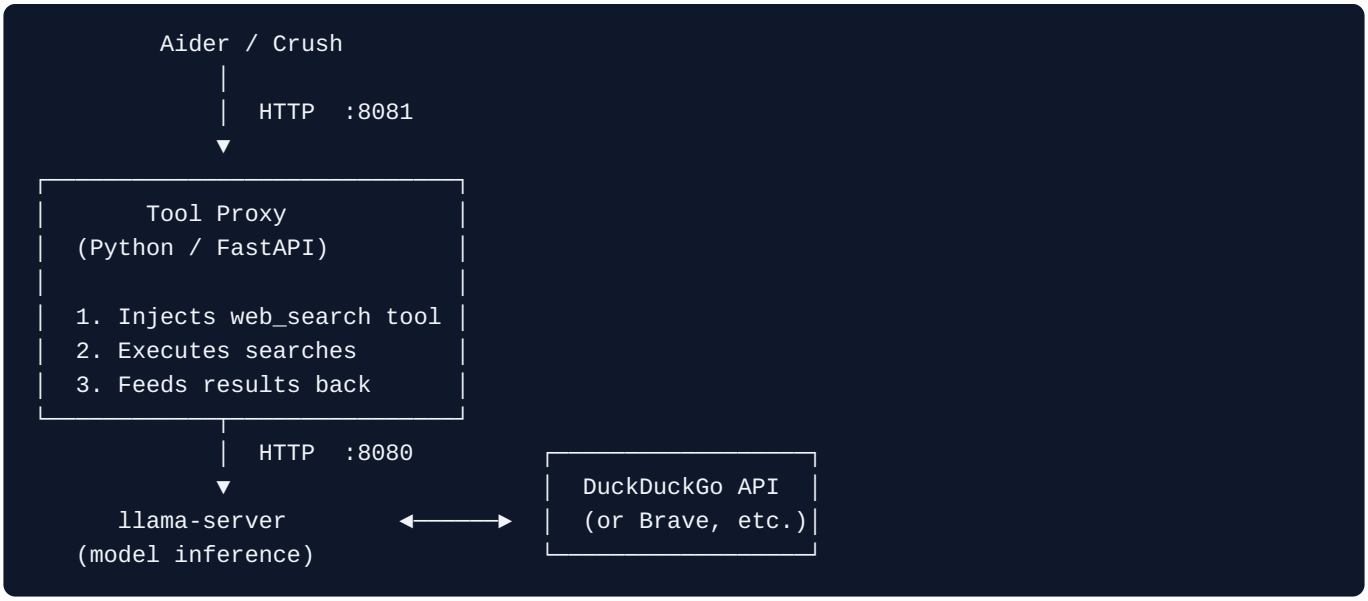
There are several ways to wire this up, but the **Tool Proxy** is the most practical for this setup because:

- It works **identically for both Aider and Crush** — both clients simply point to a different port.
- It requires **zero changes** to the Aider or Crush configuration beyond the backend URL.
- It is a bare-metal Python process — no containers, no daemons, no system-level services.
- You can swap the search backend (DuckDuckGo, Brave, Tavily, SearXNG) by editing a single function.

---

2. Architecture Overview

The proxy sits transparently between the clients and `llama-server`:



From the perspective of Aider and Crush, nothing has changed — they talk to an OpenAI-compatible endpoint. The proxy handles all the complexity of the tool call loop internally.

---

3. How Tool Calling Works

Without a proxy, a conversation ends at the model's first reply. With the proxy active, the flow becomes a multi-turn agentic loop:

```
You (in Aider or Crush): "Search online for the latest llama.cpp release."
        |
        ▼
[Proxy] forwards request to llama-server
        + injects the web_search tool definition
        |
        ▼
[llama-server] model decides to call web_search
        |  returns: { tool_call: web_search("llama.cpp latest release") }
        ▼
[Proxy] intercepts the tool call
        → executes DuckDuckGo search
        → gets top 5 results (title, URL, snippet)
        |
        ▼
[Proxy] sends results back to llama-server as a tool message
        |
        ▼
[llama-server] model reads the results, composes final answer
        |
        ▼
[Proxy] forwards final text response to Aider / Crush
```

The loop repeats up to a configurable maximum (default: 6 iterations) in case the model issues multiple consecutive tool calls.

> **NOTE**
> The model is **not forced** to always call the tool. It makes its own decision based on the user message and the system prompt. See Section 6 for details on controlling this behaviour.

---

4. Installing Dependencies

The proxy requires three Python packages. Install them into an isolated environment to avoid conflicts with the Aider installation.

Arch Linux

On Arch, the `EXTERNALLY-MANAGED` marker is propagated into `uv`-created venvs, so use `uv pip install` instead of plain `pip`:

```
uv venv ~/.local/share/llama-proxy/.venv --python 3.12
source ~/.local/share/llama-proxy/.venv/bin/activate
uv pip install fastapi uvicorn httpx duckduckgo-search
deactivate
```

Debian / Ubuntu

On Debian (≥ 12 Bookworm) and Ubuntu, the `EXTERNALLY-MANAGED` restriction applies only to the system Python and is **not** inherited by venvs, so plain `pip` works inside the venv:

```
uv venv ~/.local/share/llama-proxy/.venv --python 3.12
source ~/.local/share/llama-proxy/.venv/bin/activate
pip install fastapi uvicorn httpx duckduckgo-search
deactivate
```

> **TIP**
> Using `uv pip install` (as shown for Arch) also works on Debian/Ubuntu, so that command is safe to use on any distro.

Alternative: single global install with `uv`

```
uv tool install --python 3.12 uvicorn
uv pip install fastapi httpx duckduckgo-search
```

---

5. The Proxy — Modular Structure

Instead of a single monolithic script, the proxy is organised as a small package alongside `main.py`. The motivation is **extensibility**: adding a new tool only requires creating a new file in `tools/` and registering one line in `tools/__init__.py` — the routing logic in `main.py` never needs to change.

```
llama-proxy/
├── __init__.py          # empty package marker (kept intentionally blank)
├── config.py            # runtime constants and system prompt
├── main.py              # FastAPI app: agentic tool loop + transparent passthrough
├── requirements.txt     # minimal Python dependencies
└── tools/
    ├── __init__.py      # tool handler registry — the only file to edit when adding a tool
    └── search.py        # DuckDuckGo implementation + OpenAI tool definition
```

Create the directory structure:

```
mkdir -p ~/.local/share/llama-proxy/tools
```

`config.py`

Defines all runtime constants in one place. Changing the model host, port, search depth, or system prompt only requires editing this file.

- **`BACKEND_URL`** — address of the running `llama-server` instance (default: `http://localhost:8080`)
- **`PROXY_PORT`** — port on which the proxy listens (default: `8081`)
- **`MAX_RESULTS`** — maximum number of DuckDuckGo results returned per search (default: `5`)
- **`MAX_LOOPS`** — safeguard cap on the agentic tool call loop, preventing infinite chains (default: `6`)
- **`SEARCH_SYSTEM_PROMPT`** — system message automatically injected into any conversation that has no system prompt; instructs the model to use `web_search` for current-events or version-related questions

`tools/search.py`

DuckDuckGo search implementation. The OpenAI-format tool definition lives here, next to its handler function, so adding a new tool means adding a new file with the same two-component pattern.

- **`SEARCH_TOOL`** — OpenAI tool definition dict (`type`, `function.name`, `function.description`, `function.parameters`) for the `web_search` tool; this is what gets sent to `llama-server` so the model knows the tool exists and how to call it
- **`do_search(query)`** — executes a DuckDuckGo text search via `DDGS`, returns up to `MAX_RESULTS` results formatted as numbered entries (title + URL + snippet); returns an error string on network or API failure rather than raising

`tools/__init__.py`

The central tool registry. `main.py` imports exclusively from here and is never aware of any specific tool implementation.

- **`TOOL_HANDLERS`** — `dict[str, Callable]` mapping each tool name to its handler function; the proxy calls `TOOL_HANDLERS[name](args)` when the model issues a tool call
- **`TOOL_DEFINITIONS`** — list of OpenAI tool definition dicts, injected into every outgoing request to `llama-server`

To add a new tool: create a new file in `tools/`, then add one import and one entry to each of these two objects here.

`main.py`

FastAPI application. Imports tools exclusively through the registry — never references any specific tool implementation by name.

`POST /v1/chat/completions` — the core agentic loop:

1. Logs the incoming request (message count, last user prompt preview) and injects all registered tools and the system prompt if absent.
2. Forces `stream=false` internally so the full response can be inspected for tool calls, regardless of what the client requested.
3. Runs up to `MAX_LOOPS` iterations:
4. If the model returns a **known tool call** (registered in `TOOL_HANDLERS`), the proxy executes it, appends the result as a `tool` message, and sends the updated conversation back to `llama-server`.
5. If the model returns a **client-owned tool call** (e.g. a tool injected by Crush itself), the response is passed through immediately — the proxy must not attempt to execute tools it does not own.
6. If the model returns **no tool call**, the final answer is returned to the client.
7. If the client originally requested streaming, the resolved JSON response is re-serialised as an SSE stream via `_as_sse` before being returned.

`ANY /{path}` — transparent passthrough for all other endpoints (`/v1/models`, `/health`, etc.), forwarding requests directly to `llama-server`.

CLI flags (when run as `python main.py`):

Flag	Default	Description
`--debug`	off	Sets log level to `DEBUG` for both the proxy logger and uvicorn; shows full request metadata, tool results, and response previews
`--host`	`127.0.0.1`	Bind address
`--port`	`PROXY_PORT`	Bind port

`__init__.py`

The root `__init__.py` is intentionally empty. It exists only to mark `llama-proxy/` as a Python package, which keeps the project importable if you ever run tests or distribute it. It performs no runtime work.

`requirements.txt`

The dependency list. Only four packages are needed:

- **`fastapi`** — Web framework used by `main.py`.
- **`uvicorn`** — ASGI server that runs the FastAPI app.
- **`httpx`** — Async HTTP client used to forward requests to `llama-server`.
- **`duckduckgo-search`** — DuckDuckGo search library used by `tools/search.py`.

---

6. System Prompt & When the Model Searches

`llama-server` does not execute tool calls autonomously — it is the model itself that decides whether to issue a `web_search` call or answer directly from its training data. This decision is influenced by two factors: the **system prompt** and the **wording of your message**.

The proxy script above already injects a system prompt that nudges the model toward proactive searching. The table below summarises the resulting behaviour:

User message	System prompt active	Expected behaviour
`"cerca online l'ultima versione di llama.cpp"`	Yes / No	✅ Searches — explicit instruction overrides everything
`"what is the latest llama.cpp release?"`	Yes	✅ Likely searches — system prompt flags "latest versions"
`"what is the latest llama.cpp release?"`	No	⚠️ Uncertain — model may answer from training data
`"write a Python function to merge two lists"`	Yes	✅ Answers directly — no search needed or triggered
`"explain how transformers work"`	Yes	✅ Answers directly — well-covered by training data

Key takeaway: with the system prompt active, explicitly writing `"search online"`, `"cerca su internet"`, `"look up"`, or any equivalent phrasing in any language makes the search behaviour deterministic. The model will call `web_search` before composing its answer.

> **NOTE**
>
> Local models (Qwen3) are less reliably autonomous than cloud models at deciding *when* to search unprompted. Relying on explicit phrasing combined with the system prompt gives consistent, predictable results without any performance overhead on requests that do not need a search.

---

7. Connecting Aider and Crush to the Proxy

Aider

Point Aider to port `8081` (the proxy) instead of `8080` (llama-server directly). Add `--no-stream` because the proxy's tool loop resolves the response in full before returning it:

```
~/.local/bin/aider   --openai-api-base http://localhost:8081/v1   --openai-api-key not-
needed   --model openai/qwen3   --no-stream
```

Crush

Edit `~/.config/crush/crush.json` and change `base_url` from port `8080` to `8081`:

```
{
  "$schema": "https://charm.land/crush.json",
  "providers": {
    "llama-local": {
      "id": "local-qwen3.8",
      "name": "local-qwen3.8",
      "type": "openai-compat",
      "base_url": "http://localhost:8081/v1",
      "api_key": "not-needed",
      "models": [
        {
          "id": "local-qwen3.8",
          "name": "Qwen 3.8 Local",
          "context_window": 32768,
          "default_max_tokens": 4096,
          "can_reason": true,
          "supports_tools": true
        }
      ]
    }
  }
}
```

No other changes are needed. Crush will continue to work identically — it simply talks to the proxy instead of `llama-server` directly.

---

8. Shell Aliases & Full Stack Startup

Add the following aliases alongside the existing ones in your `~/.bashrc` or `~/.zshrc`:

```
alias llama-proxy="(cd ~/.local/share/llama-proxy && .venv/bin/python main.py)"
alias aider-web="aider --openai-api-base http://localhost:8081/v1 --openai-api-key not-needed --model openai/qwen3 --no-stream"
```

> **NOTE**
>
> No alias is needed for Crush — `crush` alone already uses the `llama-local` provider configured in `crush.json`, which already points to the proxy on port `8081`.
>
> The flags in `aider-web` are all necessary: `--openai-api-base` points aider to the proxy instead of llama-server directly, `--no-stream` is required because the proxy resolves the full response before returning it, and `--openai-api-key` is a placeholder required by aider even when not used.

The recommended startup sequence using three terminal tabs or a multiplexer such as `tmux`:

```
# Tab 1 — backend model server (unchanged)
llama-coder

# Tab 2 — search proxy
llama-proxy

# Tab 3 — your coding session
aider-web   # or: crush
```

---

9. Alternative Search Backends

The `do_search()` function in `tools/search.py` is the only part that needs to change when switching search providers. All other logic remains identical.

Backend	API Key	Privacy	Notes
DuckDuckGo (`duckduckgo-search`)	None	Medium	Default in this guide. No registration required.
Brave Search	Yes (free tier: 2,000 req/mo)	High	Higher quality results. Get key
Tavily	Yes (free tier: 1,000 req/mo)	Medium	Optimised for LLM consumption. Get key
SearXNG (self-hosted)	None	Maximum	Aggregates multiple engines locally. More complex to set up.

Technical Concept Reference & FAQs

What is the `-c` (Context Window Size) parameter?

It specifies the maximum number of tokens (1 token ≈ 3–4 characters) the model can hold in memory at once (`System Prompt` + `Chat History` + `Source Code` + `Model Output`).

KV Cache Pre-allocation: `llama.cpp` pre-allocates memory for the full context window upon startup to guarantee stability. For example, `-c 32768` reserves roughly 2–3 GB of RAM depending on the model. Keep in mind that a 3,000-line codebase already amounts to ~30,000 tokens on its own — close to the 32k limit — leaving little headroom for system prompt, chat history, and model output. For heavy code-editing sessions consider a larger context window (e.g., `-c 65536`) if your available RAM allows.

Where is the `.gguf` file specified?

The `.gguf` path is passed exclusively to `llama-server` via the `-m` flag. Neither Aider nor Crush accesses the model file directly — both communicate with the backend exclusively over HTTP.

Why use the model identifier `openai/qwen3` in Aider?

This string is a LiteLLM identifier used by Aider to configure its internal logic: * `openai/` tells Aider to send requests via the OpenAI API protocol over HTTP. * `qwen3` informs Aider of the model family capabilities, optimizing its system prompts and code-editing strategy.

What is the difference between `llama-server` and `llama-cli`?

- `llama-server` loads the model once and exposes a persistent HTTP API on a given port. It is designed to serve multiple requests over time and is what Aider and Crush connect to.
- `llama-cli` is a standalone interactive terminal session. It loads the model, runs a single conversation, and exits. It does not expose any HTTP endpoint — use it for quick manual tests without setting up a server.

Can I run multiple models at the same time?

Yes, by launching multiple `llama-server` instances on **different ports**, each loading a different model. You then point each client to the corresponding port. Keep in mind that each instance independently pre-allocates RAM for its model weights and KV cache, so available system memory is the limiting factor.

How do I verify that my GPU is actually being used?

Two methods: 1. **Check `llama-server` startup logs:** Look for a line such as `llm_load_tensors: offloaded X/Y layers to GPU` . If `X` is 0, no GPU offloading is active. 2. **Monitor VRAM in real time:** Run `watch -n 1 nvidia-smi` in a separate terminal while `llama-server` is starting. You should see VRAM usage rise by the expected amount (number of offloaded layers × VRAM per layer).

What happens when the context window fills up during a conversation?

When the total tokens (system prompt + history + new message) exceed the `-c` value, `llama-server` applies a **context shift** by default: it discards the oldest tokens in the conversation to make room for new ones. This can cause the model to "forget" earlier parts of the conversation. To disable this behavior and instead receive an explicit error (useful for debugging), pass `--no-context-shift` to `llama-server` . The solution is to either increase `-c` or start a new session.

Which model family is best suited for coding tasks?

- **Qwen3** (e.g., Qwen3-8B, Qwen3-27B) — strong reasoning capabilities with a thinking/non-thinking mode switch. Excellent for mixed coding and explanation tasks. **Recommended.**
- **Qwen2.5-Coder** — purpose-built for code generation. Valid alternative if Qwen3 is not available.
- **Llama** / **Mistral families** — solid general-purpose models, but not specifically optimized for code.

For a dedicated coding assistant (Aider or Crush in code-edit mode), the **Qwen3** family is the recommended starting point.

Performance Factors & Latency Troubleshooting

When using any frontend client (Aider, Crush) to interact with large files or codebases, you may experience long delays before receiving the first token (*Time To First Token* / TTFT). The key performance variables are:

- **Prompt Processing Overhead (TTFT):** Before generating text, `llama-server` must process and compute the attention matrix for all input tokens (System Prompt + Client Context + Target File). Reading large files forces a massive initial evaluation phase proportional to the total token count.
- **CPU Thread Tuning (`-t` Parameter):** Setting `-t` incorrectly causes severe performance degradation.
- **Rule:** Set `-t` equal to the number of **physical CPU cores** (e.g., `-t 8` on an 8-core CPU).
- **Avoid:** Setting `-t` to logical threads (Hyper-Threading / SMT) leads to CPU cache thrashing and slower prompt evaluation.
- **System Memory Bandwidth (RAM Bottleneck):** LLM CPU inference is constrained by RAM speed (GB/s), not CPU clock rate. Dual-channel DDR5 provides significantly higher token generation speeds than DDR4. Full GPU offloading (via VRAM) is orders of magnitude faster than CPU RAM.
- **Quantization Level Impact:** Heavy quantizations (`Q8_0` or `Q6_K`) require moving more data through system RAM per token compared to `Q4_K_M`.
- **Frontend Context Building:** Both Aider and Crush automatically include contextual information in every prompt. Aider sends a `tree-sitter`-generated repository symbol map; Crush builds its own context window from open files and conversation history. In large projects, this significantly inflates the token count on the first request.

Quick Diagnostic Checklist for Slow Responses

1. Check the terminal running `llama-server` for metrics:
2. `prompt eval time`: Indicates initial context processing speed.
3. `eval time`: Indicates token generation speed (tokens/sec).
4. Confirm `-t` matches your physical CPU cores (not logical threads).
5. Note that subsequent questions on the same file/context are much faster because `llama-server` reuses the prompt cache.

Common Errors & Fixes

`llama-server` exits immediately or prints "failed to load model"

Cause: Not enough system RAM to load the model weights plus KV cache at the requested context size. **Fix:** Try one or more of the following: * Switch to a lighter quantization (e.g., `Q4_K_M` instead of `Q6_K`). * Reduce the context window (e.g., `-c 8192` instead of `-c 32768`). * Use a smaller model. * If GPU offloading is active, lower `-ngl` to reduce VRAM pressure and let more layers stay in RAM.

Aider or Crush reports "Connection refused" or cannot reach the server

Cause: `llama-server` is not running, crashed silently, or is listening on a different port than the one the client expects. **Fix:** * Verify `llama-server` is running and check its terminal output for errors. * Confirm the port in the client configuration (`--openai-api-base` for Aider, `base_url` in `crush.json`) matches the `--port` value passed to `llama-server` (default: `8080`).

Model outputs garbled, repetitive, or truncated text

Cause: Chat template mismatch — the model is receiving prompts in a format it was not trained on, causing it to interpret role markers as regular text. **Fix:** * For Aider: ensure the `--model` identifier (e.g., `openai/qwen3`) matches the actual model family loaded in `llama-server`. Using a mismatched identifier applies the wrong prompt format. * For Crush: verify the `id` and `name` fields in `crush.json` reflect the actual loaded model. * As a quick test, run `llama-cli` directly in interactive mode (`-cnv`) to isolate whether the issue is in the model or in the client layer.

`CUDA error: out of memory` or process is killed by the OS

Cause: `-ngl` is set too high and the offloaded layers exceed available VRAM, or the KV cache allocation pushes over the limit. **Fix:** Reduce `-ngl` incrementally (e.g., by steps of 4) and restart `llama-server`. Monitor VRAM with `nvidia-smi` at startup to find the safe ceiling for your hardware.

`llama-server` starts without errors but only uses the CPU (GPU is idle)

Cause: `llama.cpp` was compiled without CUDA support — the `GGML_CUDA=ON` flag was not passed to CMake, or the wrong binary from a previous CPU-only build is being executed. **Fix:** * Confirm you are running the binary from the CUDA-enabled build (`build/bin/llama-server` inside the directory where you ran `cmake -B build -DGGML_CUDA=ON`). * Delete the `build/` directory entirely and recompile from scratch following the CUDA build steps in the GPU Acceleration section. * Verify with `llama-server` startup logs: a CUDA-enabled build will print GPU device information (name, total VRAM) before loading the model.

`aider` command not found after installation

Cause: `uv` installs binaries into `~/.local/bin`, which is not in the shell `PATH` by default on all systems. **Fix:** Add the following to your `~/.zshrc` or `~/.bashrc` and reload the shell:

```
export PATH="$HOME/.local/bin:$PATH"
source ~/.zshrc  # or source ~/.bashrc
```

Verify with `which aider` — it should return a path under `~/.local/bin/`.

Crush or Aider stops generating mid-response or returns empty output

Cause A (Crush): `default_max_tokens` in `crush.json` is too low for the requested response length. **Fix:** Increase `default_max_tokens` (e.g., from `4096` to `8192`) in `~/.config/crush/crush.json`. **Cause B (context overflow):** The total prompt length has reached the `-c` limit and the model has no tokens left to generate output. **Fix:** Start a fresh session, reduce the number of files added to context, or increase `-c` (and update `context_window` in `crush.json` accordingly).

Very high RAM usage at idle, before sending any prompt

Cause: This is expected behavior. `llama.cpp` pre-allocates the full KV cache for the requested context window (`-c`) at startup, in addition to loading model weights. This is by design to guarantee stability and avoid runtime memory allocation failures. **Fix:** If memory is tight, reduce `-c` to a smaller value. The RAM reserved for the KV cache scales linearly with the context window size.